Project 1 — AgentDesk

An AI Research & Knowledge Workspace

Think:

> **ChatGPT + Perplexity + NotebookLM + Cursor-style tool UI**

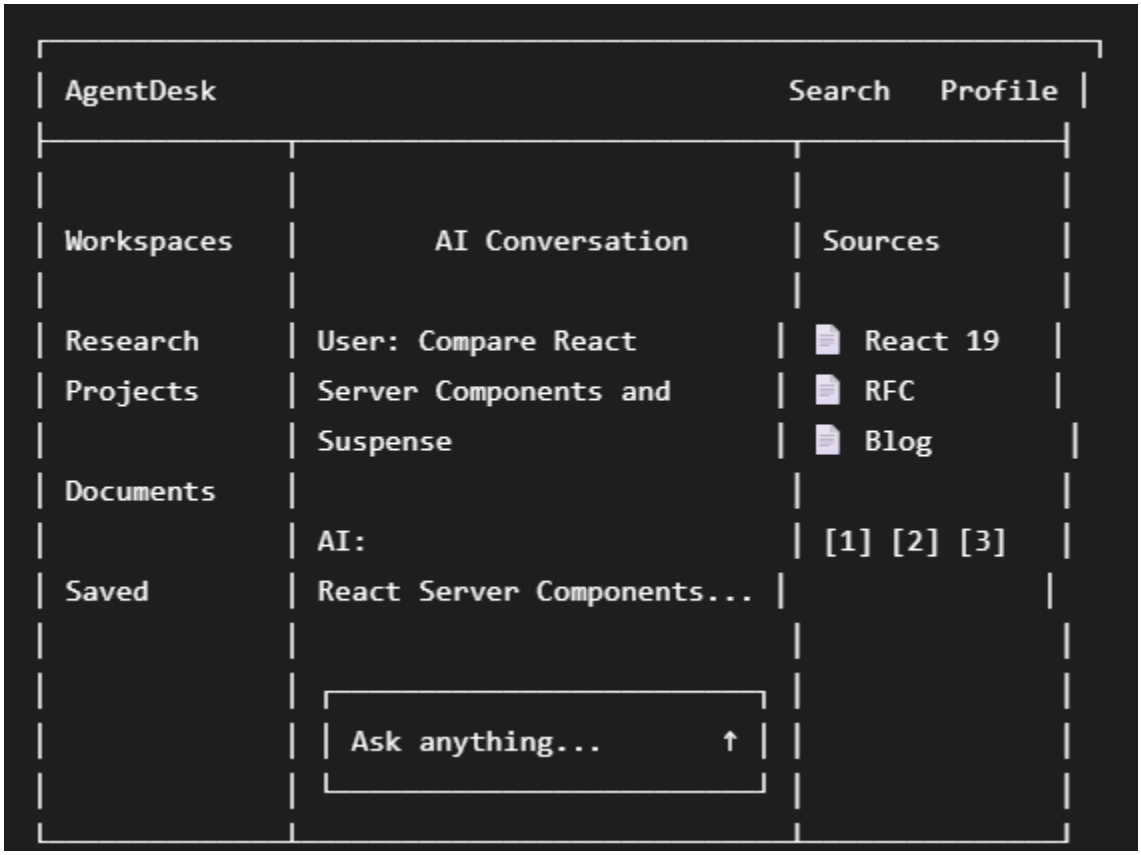
but focused on **knowledge work**.

The user can upload documents, connect sources, ask questions, create research tasks, compare documents, generate reports and interact with AI-generated UI.

AI Research & Knowledge Workspace

Product idea

Imagine the user opens:



But the AI doesn't just answer.

It can:

Search documents
Search web
Read document
Compare documents
Extract structured data
Generate table
Generate chart
Create summary
Create research plan
Ask clarification
Cite sources
Save findings

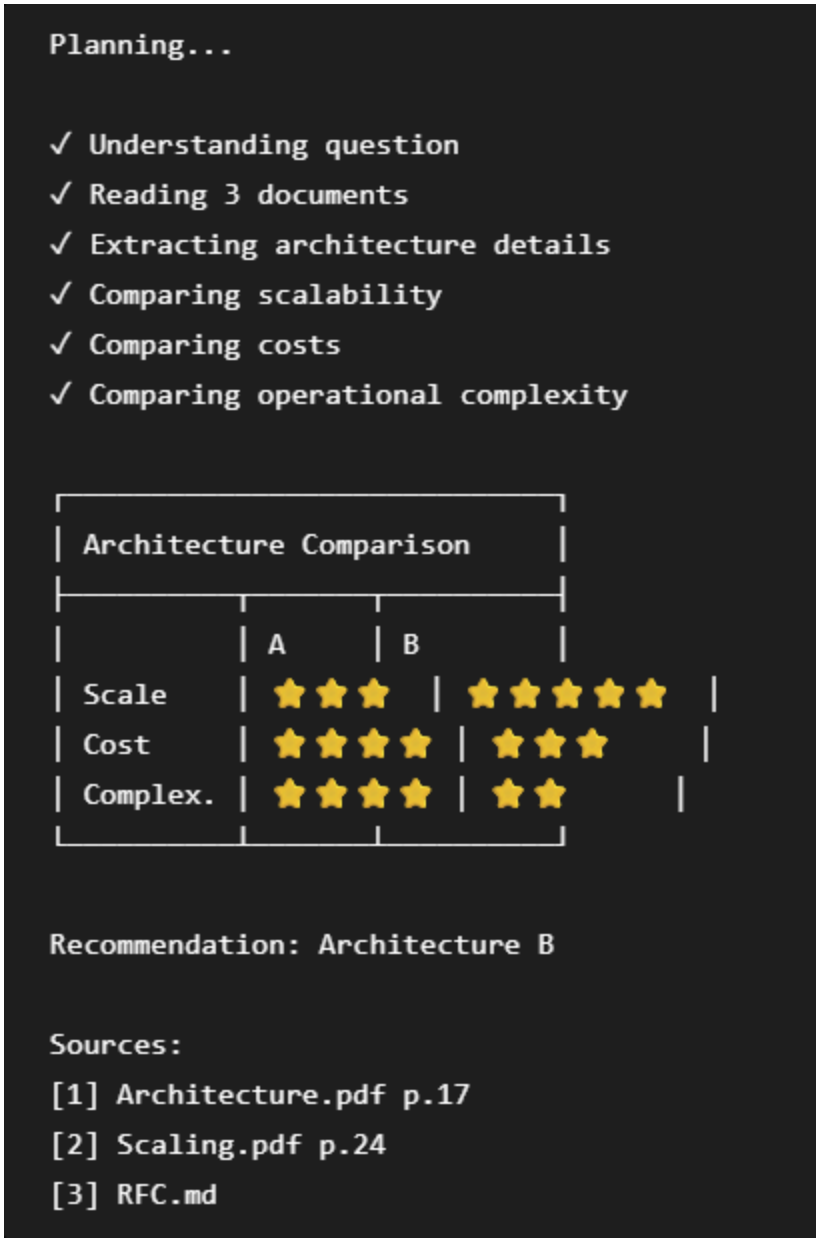
The current AI UI ecosystem already treats tool calls and different message parts as first-class UI concerns rather than simply rendering a string response.

AgentDesk — Core AI workflow

User:

> "Compare these three architecture documents and tell me which architecture is best for a 10M-user SaaS application."

The agent might produce:



AgentDesk feature set

1. Streaming chat

Absolutely mandatory.

Do not wait for:

response = complete_response

Instead:

AI starts generating
↓
token/chunk
↓
React
↓
screen updates

The current AI SDK chatbot architecture explicitly uses streaming to progressively update the UI.

But here's the important part:

Don't hide the streaming implementation.

Implement the frontend stream handling yourself initially.

Your Python backend:

FastAPI
↓
SSE
↓
React

Frontend:

fetch()
↓
ReadableStream
↓
TextDecoderStream
↓
SSE parser
↓
message reducer
↓
React

This single feature teaches you:

- Streams

- Async iteration
- Microtasks
- Event loop
- Web Streams
- AbortController
- Backpressure concepts
- React rendering
- state updates

---

2. Tool-call visualization

This is one of the features that will make the project look like a real AI product.

Instead of:

AI: I searched your documents...

show:

```
┌──────────────────────────────────────┐
│ 🔎 Searching documents               │
│                                      │
│ React-Architecture.pdf               │
│ Scaling.pdf                          │
│ RFC.pdf                              │
│                                      │
│ ✓ 17 relevant sections found         │
└──────────────────────────────────────┘
```

Then:

```
┌──────────────────────────────────────┐
│ 📊 Comparing architectures           │
│                                      │
│ ████████████████████░░ 78%           │
└──────────────────────────────────────┘
```

Then:

```
┌──────────────────────────────────────┐
│ ✓ Analysis complete                  │
│                                      │
```

```
│ [View comparison]              │
└──────────────────────────────────┘
```

This teaches you to model messages as:

```
type MessagePart =
  | TextPart
  | ToolCallPart
  | ToolResultPart
  | CitationPart
  | ChartPart
  | TablePart
  | ArtifactPart;
```

That is **advanced TypeScript + React architecture**.

---

3. Generative UI

This should be one of your flagship features.

Instead of the AI returning:

Here are the results in a table...

it returns structured information:

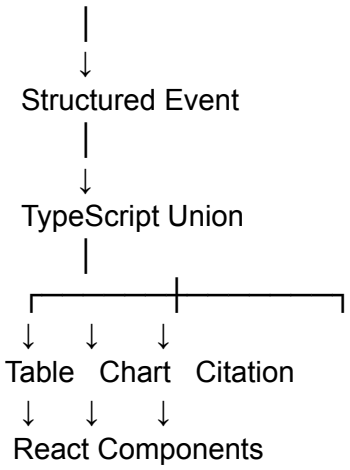
```
{
  "type": "comparison_table",
  "data": {}
}
```

React decides:

```
comparison_table → ComparisonTable
chart → Chart
document → DocumentViewer
citation → CitationCard
search → SearchResults
approval → ApprovalPanel
```

Architecture:

```
        AI
```



This is exactly the direction modern AI UI tooling is moving toward: tool calls and message parts can drive richer UI instead of only text.

---

4. Citation system

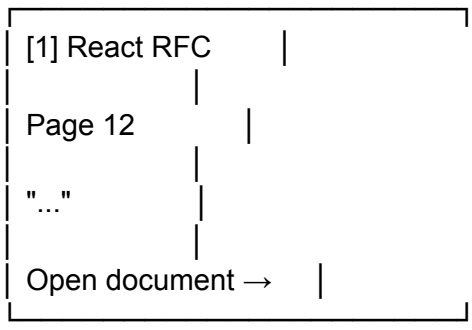
This should be **extremely good**.

When AI says:

> React's rendering model is interruptible...

show:

React's rendering model is interruptible.[1]



Clicking [1] opens the document to the exact location.

This teaches:

- URL/state synchronization
- virtualization
- document rendering
- overlays
- accessibility
- state management
- deep linking

---

5. Document viewer

Upload:

PDF
Markdown
TXT
DOCX
CSV
JSON

Then:

Chat
+
Document Viewer
+
Source Highlighting

The AI can say:

> This conclusion comes from page 37.

The UI automatically scrolls to page 37.

---

6. Research mode

Have two modes:

Chat mode

Fast:

Question
↓
Agent
↓
Answer

Research mode

Long-running:

Question
↓
Plan
↓
Search
↓
Read
↓
Compare
↓
Analyze
↓
Draft
↓
Review
↓
Final report

The frontend should show this as a state machine:

IDLE
↓
PLANNING
↓
SEARCHING
↓
READING
↓
ANALYZING
↓
WRITING
↓
REVIEWING

↓
COMPLETE

This gives you practical experience with **state machines**, not just `isLoading`.

---

7. Pause / Resume / Cancel

This is important.

User starts research:

Researching...

Then:

[Pause]

or:

[Cancel]

Then:

Paused

[Resume]

Now you're dealing with:

long-running AI workflows
abort signals
stream cancellation
state persistence
reconnection

That is senior-level frontend work.

---

8. Optimistic interactions

Example:

User:

> Save this research report.

Don't wait for backend.

Immediately:

✓ Saved

Then reconcile with server.

If backend fails:

⚠ Failed to save

Retry

You learn:

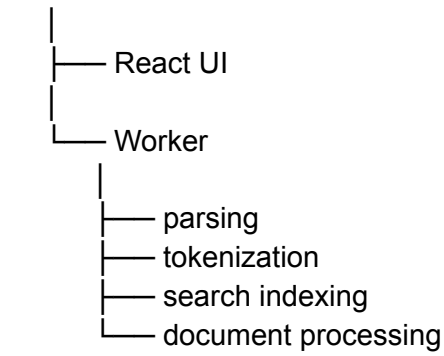
- optimistic state
- rollback
- server state vs client state
- race conditions

---

9. Web Worker

This is where your JavaScript topics become practical.

Large PDF/document processing:



Now you understand:

- Web Workers
- structured clone
- transferable objects
- main-thread performance
- rendering responsiveness

---

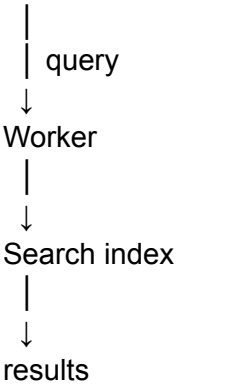
10. Search index in browser

Build a local search index.

Example:

1000 documents
100,000 chunks

Search shouldn't block React.



Now your project demonstrates actual browser architecture.

---

11. Service Worker

Add offline capabilities.

For example:

Previously opened documents
↓
Service Worker
↓
Cache

User loses internet:

Offline

✓ Previously opened research
✓ Saved conversations
✓ Cached documents

⚠ AI unavailable

That's a much better reason to learn Service Workers than studying the API theoretically.

---

12. Shared Worker

Use it for cross-tab state.

Open:

Tab 1 → AgentDesk
Tab 2 → AgentDesk
Tab 3 → AgentDesk

All tabs share:

connection
notifications
workspace state

You can learn SharedWorker naturally.

---

13. Advanced TypeScript architecture

Create:

```
type AgentEvent =
  | {
      type: "text_delta";
      text: string;
    }
  | {
      type: "tool_started";
      tool: ToolName;
      callId: string;
    }
  | {
      type: "tool_completed";
      callId: string;
      result: unknown;
    }
  | {
      type: "citation";
      citation: Citation;
    }
  | {
      type: "artifact";
      artifact: Artifact;
    }
  | {
      type: "error";
      error: AgentError;
    };
```

Then use:

Generics
Conditional Types
infer
Mapped Types
Template Literal Types
Discriminated Unions
Utility Types
Variance

This becomes your **TypeScript playground disguised as a real product**.